Building Dynamic, Intelligent Web Based Solutions with ASP.NET

I T W P 2 3 0 0

Paul C. Sbraccia, MA

Macomb Community College

2012

Version 3.4 2017

Table of Contents

First Day Activities **1**

Homework 1 ..... **11**

Homework 2 ..... **27**

Homework 3 ..... **37**

Homework 4 ..... **43**

Homework 5 ..... **55**

An Online eBook Book Store

Project 1 ..... **71**

Project 2 ..... **85**

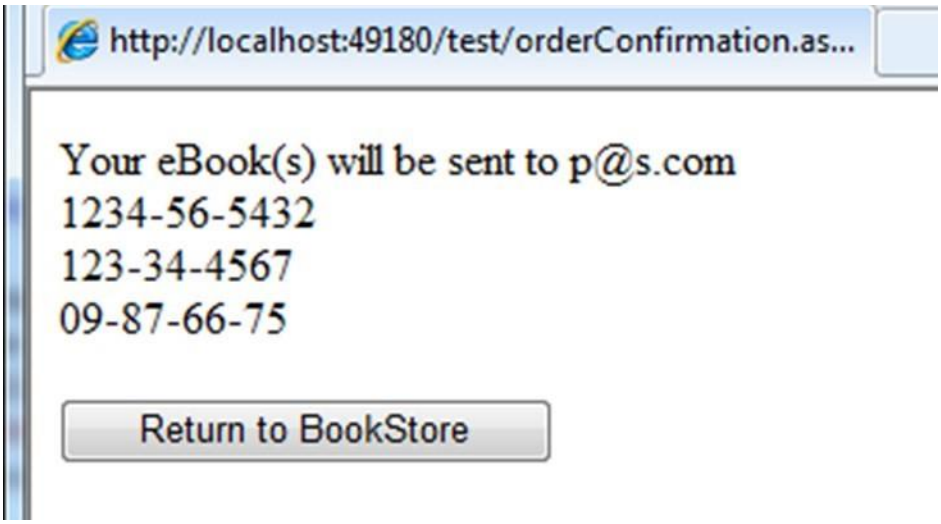
Project 3 ..... **95**

Project 4 ..... **107**

Project 5 ..... **115**

ITWP 2300

BUILDING DYNAMIC, INTELLIGENT WEB BASED SOLUTIONS WITH ASP.NET



PROJECT 4

In this unit we will modify the Shopping Cart page with our first bit of data mining. We will add the ability to show additional books based on the purchasing prerogatives of people who purchased a selected book.

In this section, we are going to expand the capabilities of the shopping cart to include a list of recommended reads based on "people who bought this book also purchased the following. We are going to interrogate the History table where we store email and ISBN of all purchases.

Preparations:

Make a copy of your BobsBookNook3 folder saving it as BobsBookNook4.

The Shopping Cart – Creating History.

Before we can do much with the book, we need to know who we are billing! In an actual application, we need to collect first name, last name, email address, shipping address, and any other information that our financial institution would need. This information is covered in the eCommerce course, so we will only create a pay button that is active, but not connected to any outside service.

To prepare for the creation of History, we need to make sure the table exists. Back in Admin.aspx.cs, we add one more call to checkTables:

```
checkTables(txtOwnerID.Text + "HISTORY","CREATE TABLE " + txtOwnerID.Text + "HISTORY
(EMAIL CHAR(64), ISBN CHAR(14))");
```

Be sure to execute Admin.aspx to create table History.

ShoppingCart Updates

Currently our shoppingcart.Checkout button looks like this:

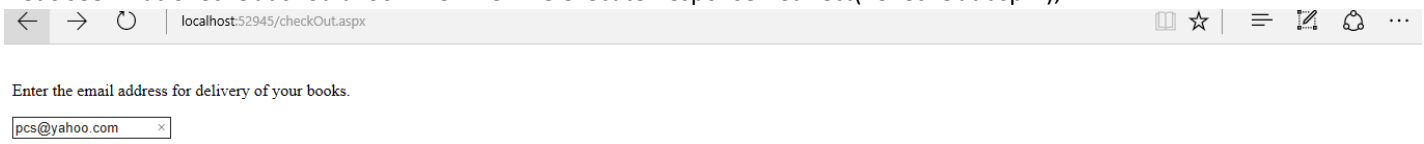
```
protected void btnCheckOut_Click(object sender, EventArgs e)
{

     try
        {
            string sqlCommand;
            string table2 = Session["tempUserID"].ToString();

            sqlCommand = "DROP TABLE " + table2;
            myDatabaseConnection.executeSQL(sqlCommand, ref gvShoppingCart, ref lblErrorMessage);
            gvShoppingCart.Visible=false;
            Session["tempUserID"] = null;
        }

        catch { }

        Response.Redirect("Default.aspx");
}
```

We make the following change in shoppingCart in anticipation of inclusion of the CheckOut.aspx page.

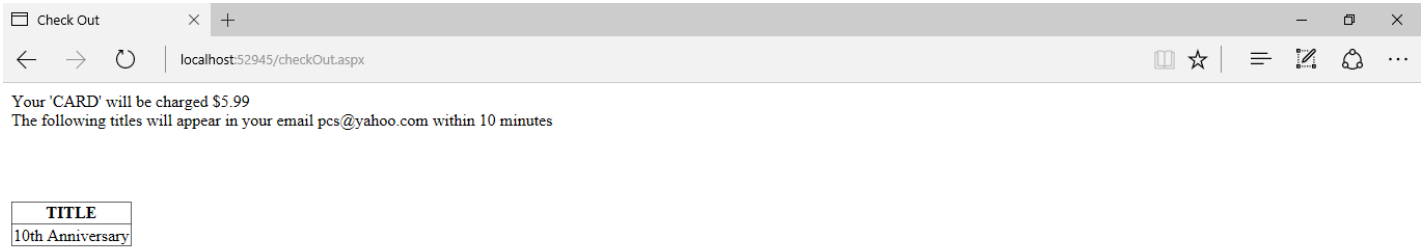
```
protected void btnCheckOut_Click(object sender, EventArgs e)
{

    //try
        //{
        //    string sqlCommand;
        //    string table2 = Session["tempUserID"].ToString();

        //    sqlCommand = "DROP TABLE " + table2;
        //    myDatabaseConnection.executeSQL(sqlCommand, ref gvShoppingCart, ref
lblErrorMessage);
        //    gvShoppingCart.Visible=false;
        //    Session["tempUserID"] = null;
        //}

        //catch { }

        //Response.Redirect("Default.aspx");
     Response.Redirect("CheckOut.aspx");
 }
```

We want to send this to a "CheckOut" page where we will capture the email address from the user and generate a validation of charges and what we are shipping.

Let's see what CheckOut should look like when we execute Response.Redirect("CheckOut.aspx");



And after the email address is entered (txtEmail.autoPostback is set to true) the following screen will replace the original.



Start by moving CheckOut.aspx from PageTemplates..

We will build the page step by step so that we can see how to use a session variable to identify the table containing our individual sales information and retrieve that information from the temporary table, eventually deleting this table when we have completed our purchase.

Capture the Session Variable

All code for this page will be located within Page_Load. Make sure you are **`using` System;** and **`using` `System.Web.UI.WebControls;`** You will need to add **`using` `System.Web.UI.WebControls;`**

Add **lblMessage** and **gvISBN** to the CheckOut.cs page.

Insert the following code in Page_Load so we can validate that we still know the "magic" file number.

```
protected void Page_Load(object sender, EventArgs e)
{
        string sqlCommand;
        string table2 = Session["tempUserID"].ToString();
        sqlCommand = "SELECT * FROM " + table2;
        myDatabaseConnection.executeSQL(sqlCommand, ref gvISBN, ref lblMessage);
}
```

Showing all the ISBNs from the unique shopping cart.

- Add a textbox named **txtEmail**. *Note: You can use txtEmail or txtEMail, just be consistent.*
- Edit the text attribute of lblMessage to read "**Please enter the email address for delivery of your books.**"
- Set **txtEMail.AutoPostback = true;**

- *Ed Note: Knowing that you understand how to validate email (from homework 4) we are going to conserve time and not require validation here.*
- Set the "validation" [the completion of the textbox] within if(!IsPostBack) and place the code we currently have within an if(IsPostBack) block.

Edit Page_Load to look like the following:

```
protected void Page_Load(,,,
{
    if (!IsPostBack)
    {
        lblMessage.Text = "Please enter the email address for delivery of your books.";
        txtEmail.Visible = true;
        txtEmail.Text = "";
    }
    if (IsPostBack)
    {
        lblMessage.Visible = false;
        lblMessage.Text = "";
        txtEmail.Text = "the following books will be sent to " + txtEmail.Text + " within
    10 minutes.";

        string sqlCommand;
        string table2 = Session["tempUserID"].ToString();
        sqlCommand = "SELECT * FROM " + table2;
        myDatabaseConnection.executeSQL(sqlCommand, ref gvISBN, ref lblMessage);
    } //End of IsPostBack
```

```
}
```

Executing the program after the modifications above, the screen should look like this:

Upon entering the email and either pressing the tab key or [Enter], an auto post back will be executed causing the page to reload currently showing the ISBNs of chosen books.

Looking at the screen shot above, it is not clear which books are going to be forwarded to p@s.com. There is a solution, since we have all the data in a table or three.

It is important to remember that our customers need clarification as they are not talking to a human and require the same level of certainty. It will not be hard at all for us to mine the data that they need.

If we were to simply show the title and author, most folks would be happy. It might look like this:

The following books will be sent to p@s.com in 10 minutes:

```
Silent Intent              Stuart Woods
The Murder Book            George Williams
Underdog, an Adventure     Martha Stuart
```

We could easily include the image as we have in previous pages, but for the sake of learning, let us stop here.

We NEED an additional GridView object added to our page. Please add gvDisplay below gvISBN and set gvISBN.Visible to false.

First Things First

Let us add the email address and ISBN to the history table.

Insert a reference to a global variable ownerID, and pull the value from App_Data.

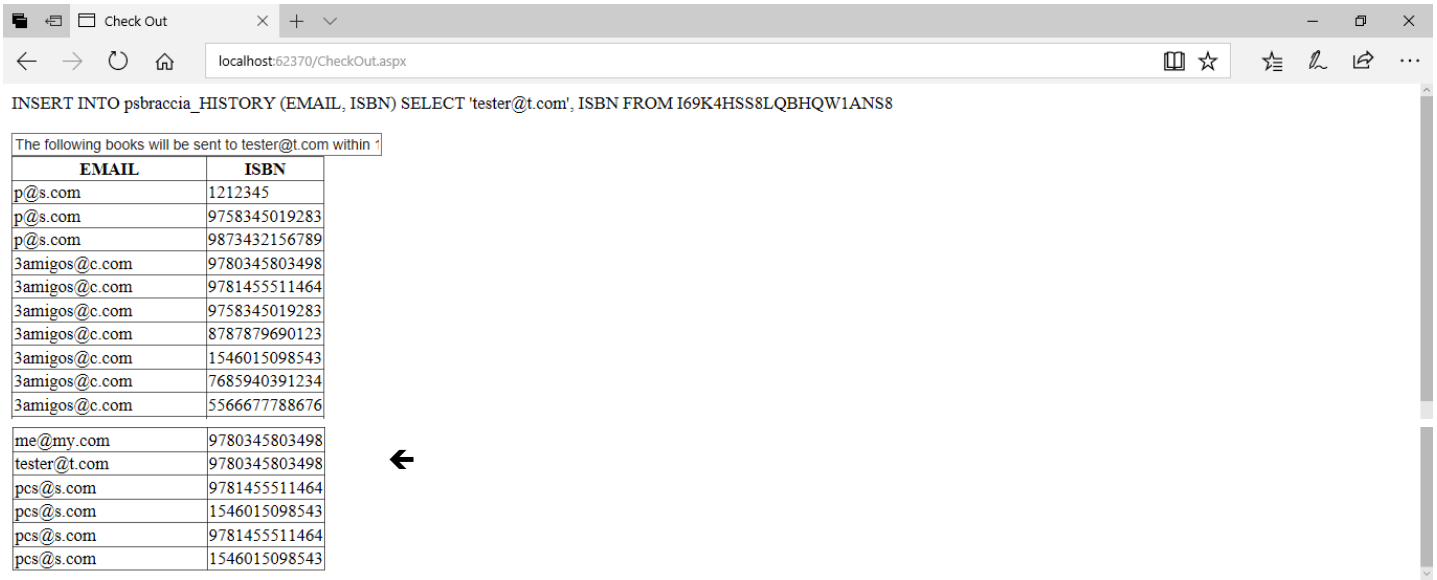
```
string ownerID;
```

```
protected void Page_Load(object sender, EventArgs e)
{
        ownerID = getOwnerID.getID(Server.MapPath("~/App_Data/"));
        :
        :
        :
```

Next, let us add the email and ISBNs to the history table:

Ed Note: This code is placed within the IsPostBack code block. There is only one IsPostBack and only one !IsPostBack.

```
if (IsPostBack)
{
        //maintain email address for history
        string historyEMail = txtEmail.Text;
        //Re-populate the txtbox with the message that says these are to be sent to you.
        txtEmail.Text = "The following books will be sent to " + txtEmail.Text + " within 10 minutes.";

        string sqlCommand;
        string table2 = Session["tempUserID"].ToString();
        sqlCommand = "SELECT * FROM " + table2;
        myDatabaseConnection.executeSQL(sqlCommand, ref gvISBN, ref lblMessage);

        //send purchases to History
        sqlCommand = "INSERT INTO " + ownerID + "HISTORY (EMAIL, ISBN) SELECT '" +
historyEMail + "', ISBN FROM " + table2;
        Response.Write(sqlCommand); //Make sure to delete me prior to final submission
        myDatabaseConnection.executeSQL(sqlCommand, ref gvDisplay, ref lblMessage);

        //show contents of history just for validation.  To be removed!!!
        sqlCommand = "SELECT * FROM " + ownerID + "HISTORY";
        myDatabaseConnection.executeSQL(sqlCommand, ref gvDisplay, ref lblMessage);
}
```



Yes, there has been a lot of testing. Remember, we are viewing the **History table** and the new addition is WAY at the bottom.

Comment out the following:

```
//show contents of history
//    sqlCommand = "SELECT * FROM " + ownerID + "HISTORY";
//    myDatabaseConnection.executeSQL(sqlCommand, ref gvDisplay, ref lblMessage);
```

We know that the History table was updated, so we can now proceed to filling **gvDisplay** with the expanded information.

The code necessary to display the title, author and ISBN is as follows:

```
//show contents of history
//     sqlCommand = "SELECT * FROM " + ownerID + "HISTORY";
//     myDatabaseConnection.executeSQL(sqlCommand, ref gvISBN, ref lblMessage);
try
{
      string table1 = ownerID + "BOOKS";
      sqlCommand = "SELECT TITLE, AUTHOR, " + table1 + ".ISBN from " + table1 + " INNER
JOIN " + table2 + " ON " + table1 + ".ISBN = " + table2 + ".ISBN ";

      myDatabaseConnection.executeSQL(sqlCommand, ref gvDisplay, ref lblMessage);

}
catch
{
      Response.Redirect("Default.aspx");
}
```

Do set the visible attribute of gvISBN to false and make sure there is no errant code setting gvISBN.Visible to true.

How Much is that Doggie in the Window?

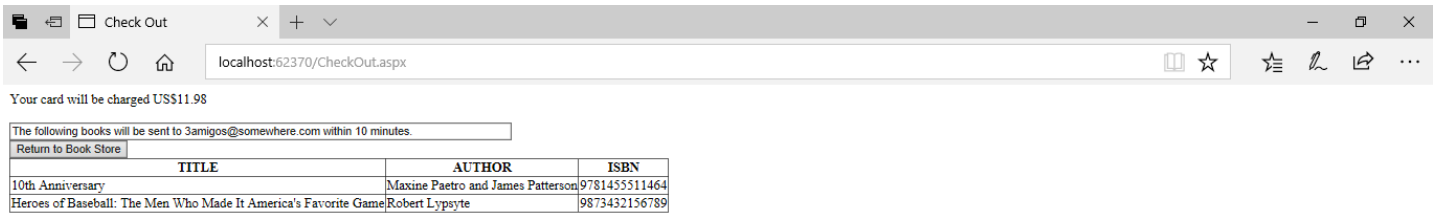
We can simply take the row count in gvISBN and multiply that by 5.99 to determine final price.

We can actually accomplish this with one statement. (The success really depends on our placement as we need to make sure gvDisplay is populated.

Shown here is just part of the IsPostBack code block.

```
string sqlCommand;
string table2 = Session["tempUserID"].ToString();
sqlCommand = "SELECT * FROM " + table2;
myDatabaseConnection.executeSQL(sqlCommand, ref gvISBN, ref lblMessage);

//you will need to add the following
gvISBN.Visible = false;

Response.Write("Your card will be charged US$" +
Math.Round(Convert.ToDecimal(gvISBN.Rows.Count * 5.99), 2) + "<br />");

//send purchases to History
```

```
sqlCommand = "INSERT INTO " + ownerID + "HISTORY (EMAIL, ISBN) SELECT '" + historyEMail + "',
ISBN FROM " + table2;
//Response.Write(sqlCommand); //Make sure to delete me prior to final submissionsubmission
myDatabaseConnection.executeSQL(sqlCommand, ref gvDisplay, ref lblMessage);
```



Clean-up Isle 4

The only thing left to do is code a Return to Bookstore button which not only redirects the individual back to Bobs Book Nook, but does so after deleting the random table we have been using. Rather than creating yet another global variable, it is more efficient to simply copy the 4 lines from Page_Load and modify the sqlCommand line, resetting the Session Variable, and returning.

```
protected void btnReturn_Click(…
{
    string sqlCommand;
    string table2 = Session["tempUserID"].ToString();
    sqlCommand = "DROP TABLE " + table2;
    myDatabaseConnection.executeSQL(sqlCommand, ref gvISBN, ref lblMessage);
    Session["tempUserID"] = null;
    Response.Redirect("Default.aspx");
}
```

You MAY want to consider making the return button Enabled button to be false until the end of the IsPostBack code.

Morning Coffee

Sitting around morning coffee just before work, the following question is posted. "Our images are really too big… is there any way we can make them smaller and more unified?"

Mary replies "We just covered that in my C# class last night! Give me 10 minutes and I will let you know if I can get it going."

Working in **Admin.aspx.cs** where the image is uploaded to the server, Mary presented the following code change to her group. They approved, and so do I.

```
protected void btnLoadImages_Click(…
{
//Clear list and label
    lstHidden.Items.Clear();
    //check to see if you have selected a file
    if (!FileUpload1.HasFiles)
    {
       //if not, error message is displayed and focus set to fileupload
       lstProgress.Items.Add("Browse for files and select");
       lstProgress.Items.Add("all images from the directory."); FileUpload1.Focus();
    }
    else
    {
```

```
        lstProgress.Items.Add("Process JPG Files.");
        foreach (HttpPostedFile uploadedFile in FileUpload1.PostedFiles)
        {
            //if the file exists on the server, it will simply be overwritten.
            //      string fileName = Path.GetFileName(uploadedFile.FileName);
            //uploadedFile.SaveAs(Server.MapPath("~/Img/") + fileName);
            //lstHidden.Items.Add(String.Format("{0}", fileName));

    System.Drawing.Image bm = System.Drawing.Image.FromStream(uploadedFile.InputStream);
            //      Response.Write(bm.Width + "   " + bm.Height + "   "  + "<HR>");
            bm = ResizeBitmap((Bitmap)bm, 100, 150); /// new width, height
            int len = uploadedFile.FileName.ToString().Length;
            int x = uploadedFile.FileName.ToString().LastIndexOf('\\');
            string newFileName = uploadedFile.FileName.ToString().Substring(x + 1);
            //Response.Write(newFileName + "<hr>");//TESTING ONLY
            bm.Save(Path.Combine(uploadPath, newFileName));
        }
    }
```

Add the following function to your code:

```
private Bitmap ResizeBitmap(Bitmap b, int nWidth, int nHeight)
{
      Bitmap result = new Bitmap(nWidth, nHeight);
      using (Graphics g = Graphics.FromImage((System.Drawing.Image)result))
                  g.DrawImage(b, 0, 0, nWidth, nHeight);
      return result;
}
```

Please erase all the jpg files from /Img and reload them running Admin.aspx as the default app.

Ed Note: The easiest thing to do is click the red [Clear Database] button in admin.aspx.

Run this on the School Server to verify that it works. (By selecting the new images, you are overwriting the old).

Ed Note: You may not "see" this implemented on the local machine. When you upload this project to the server, be sure to only copy /Img AFTER you have removed all the images. It would be a good thing to run Admin.aspx,click on [Clear Database], reload the jpg images AND reload the XML file.

PeopleWhoPurchasedThis

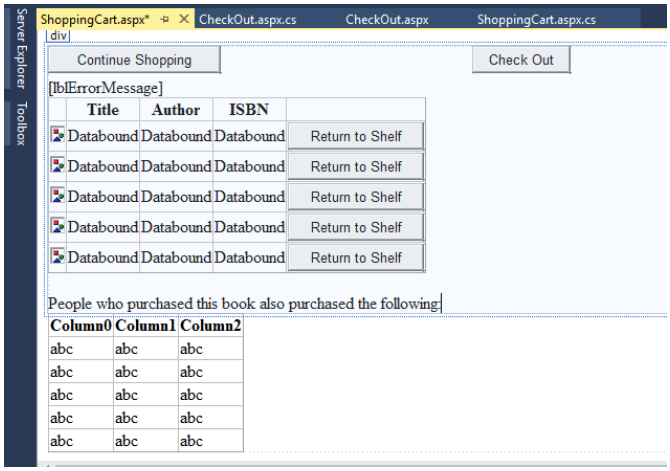
While this is not a good section title, it is descriptive of the gridview we will be adding to ShoppingCart.

We just completed populating the History table with email and ISBN. What we will do, is interrogate the history table looking for any email that is associated with the current ISBN.

We then (using an inner join) bring up all books (except the current ISBN) that everyone who purchased this book purchased.

It is not all that convoluted. It just takes patience and a good Database Manager.

Working in ShoppingCart:



- Add **gvPeopleWhoPurchasedThis** to the page.
- In order to add the **People who purchased…** text use a label and set the text attribute.

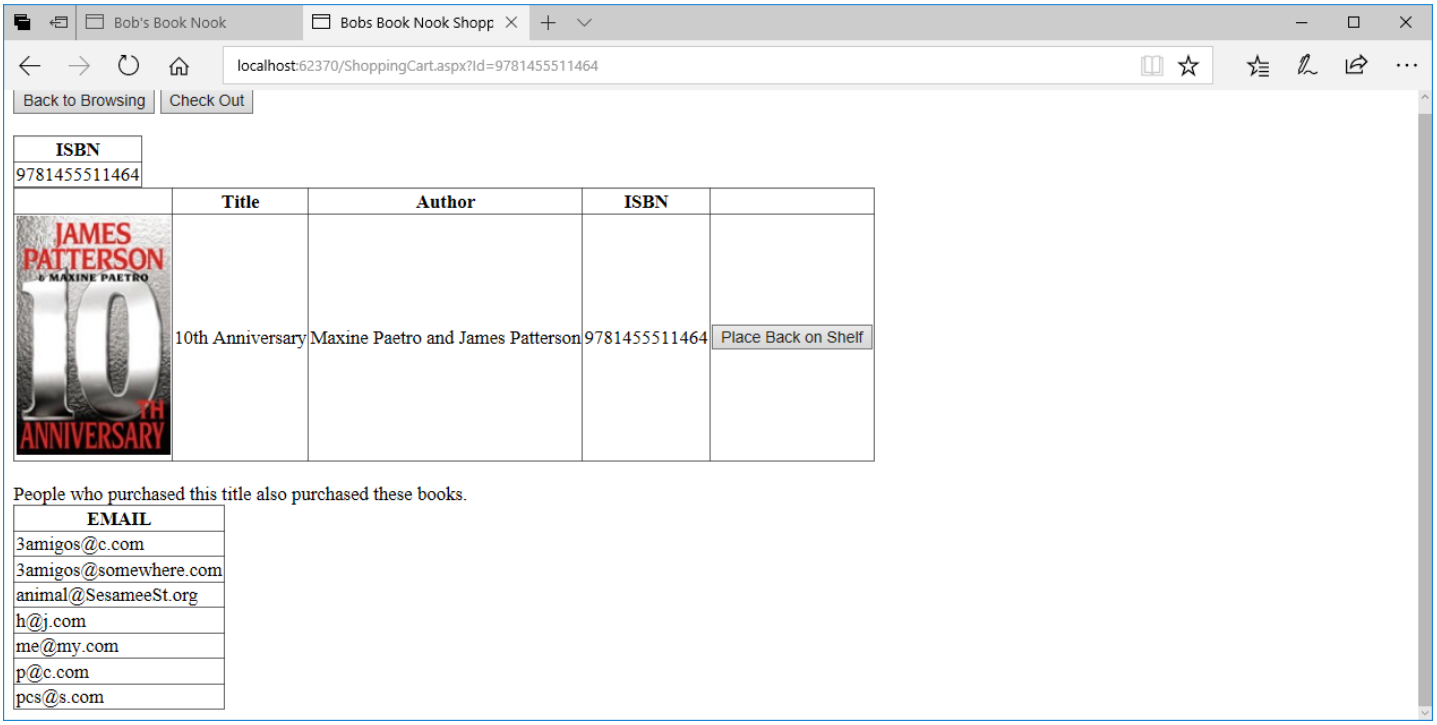
To begin the data mining, we will call **loadPurchase(Request.QueryString["id"]);** Just below the call to loadGridView() *in Page_Load*. Allow the GUI to create the method.

Ed Note: Clarification

```
protected void Page_Load(object sender, EventArgs e)
{
.
.
.
    if (!IsPostBack)
    {
        //show session variable so we know it works.

        //     Response.Write(Session["tempUserID"].ToString() + "<br/>");

        //Show ISBN of new Selection
        //      Response.Write(Request.QueryString["id"] + "<br/>");
        //Add ISBN to database table and view
        addToShoppingList(Session["tempUserID"].ToString(), Request.QueryString["id"]);
    }

     viewGvDisplay();

      loadPurchase(Request.QueryString["id"]);
}
```

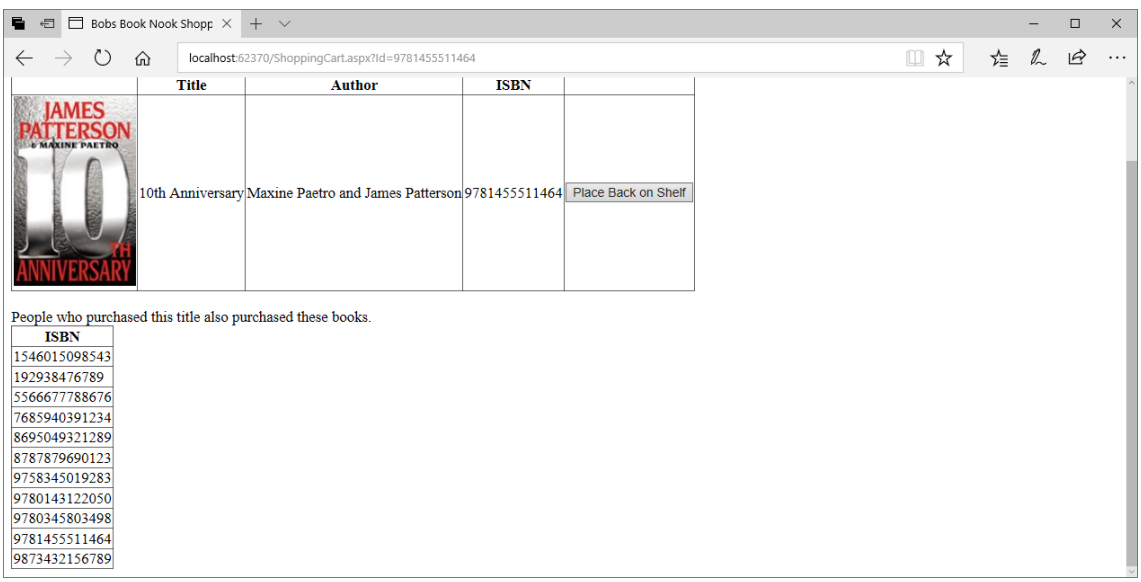
Edit the parameter list of the new method as shown below.

```
private void loadPurchase(string ISBN)
{
    //    throw new NotImplementedException();
    string sqlCommand;
    //    throw new NotImplementedException();
    sqlCommand = "SELECT DISTINCT EMAIL FROM " + ownerID + "HISTORY WHERE ISBN = '" +
ISBN + "'";

    myDatabaseConnection.executeSQL(sqlCommand, ref gvPeopleWhoPurchasedThis, ref
lblErrorMessage);

 }
```

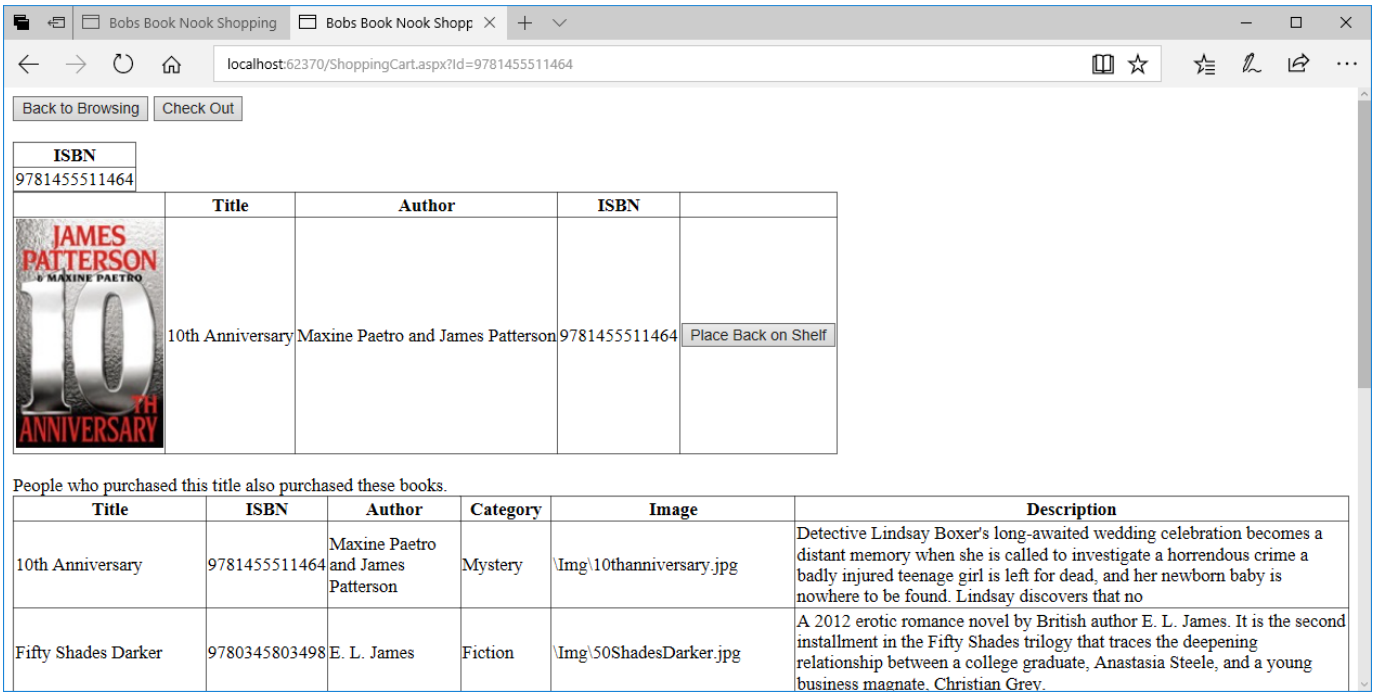


Here are each of the SQL statements and the resulting gridview:

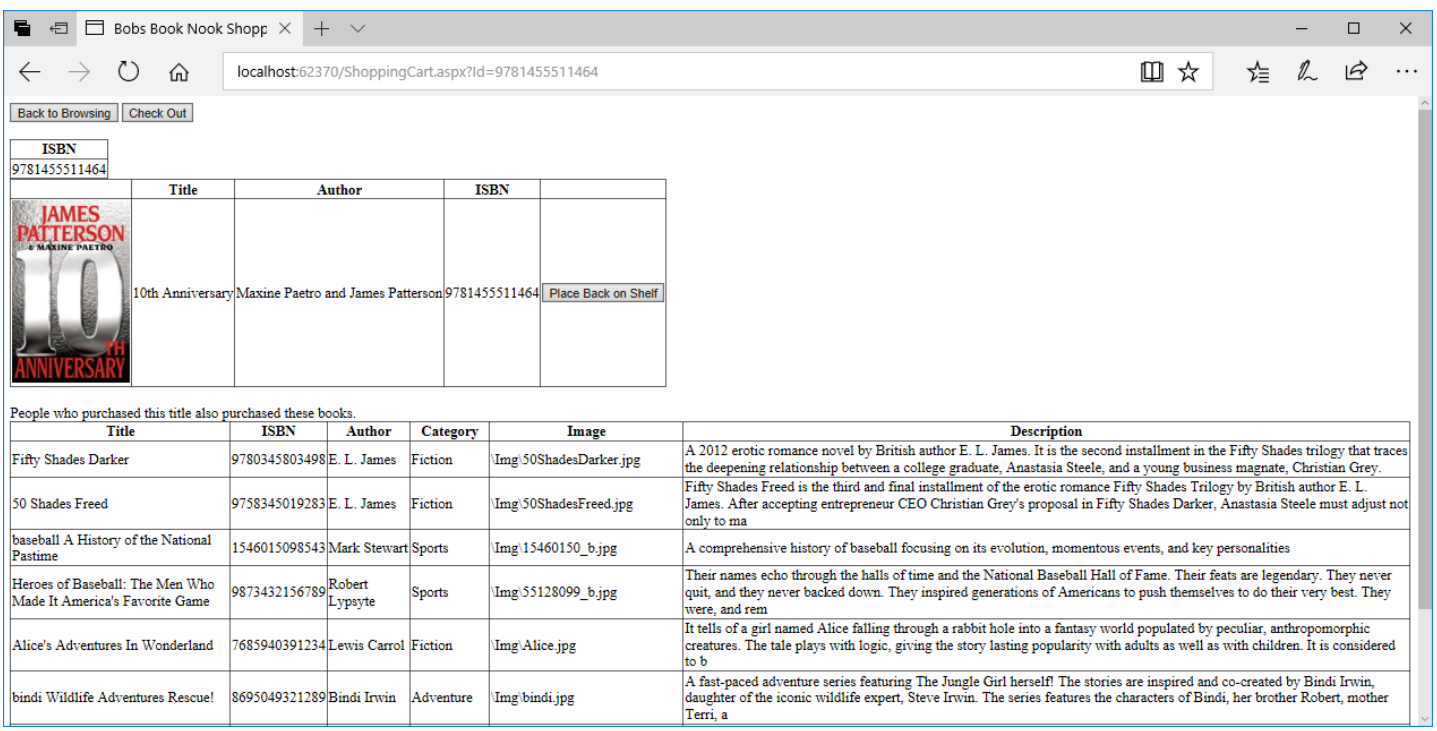
```
sqlCommand = "SELECT DISTINCT ISBN FROM " + ownerID + "HISTORY WHERE " + ownerID + "HISTORY.EMAIL IN (SELECT DISTINCT " + ownerID + "HISTORY.EMAIL FROM " + ownerID + "HISTORY WHERE " + ownerID + "HISTORY.ISBN = '" + ISBN + "')";
```




```
  sqlCommand = "SELECT * FROM " + ownerID + "BOOKS WHERE " + ownerID + "BOOKS.ISBN IN (SELECT DISTINCT ISBN FROM " + ownerID + "HISTORY WHERE " + ownerID + "HISTORY.EMAIL IN (SELECT DISTINCT " + ownerID + "HISTORY.EMAIL FROM " + ownerID + "HISTORY WHERE " + ownerID + "HISTORY.ISBN = '" + ISBN + "'))";
```

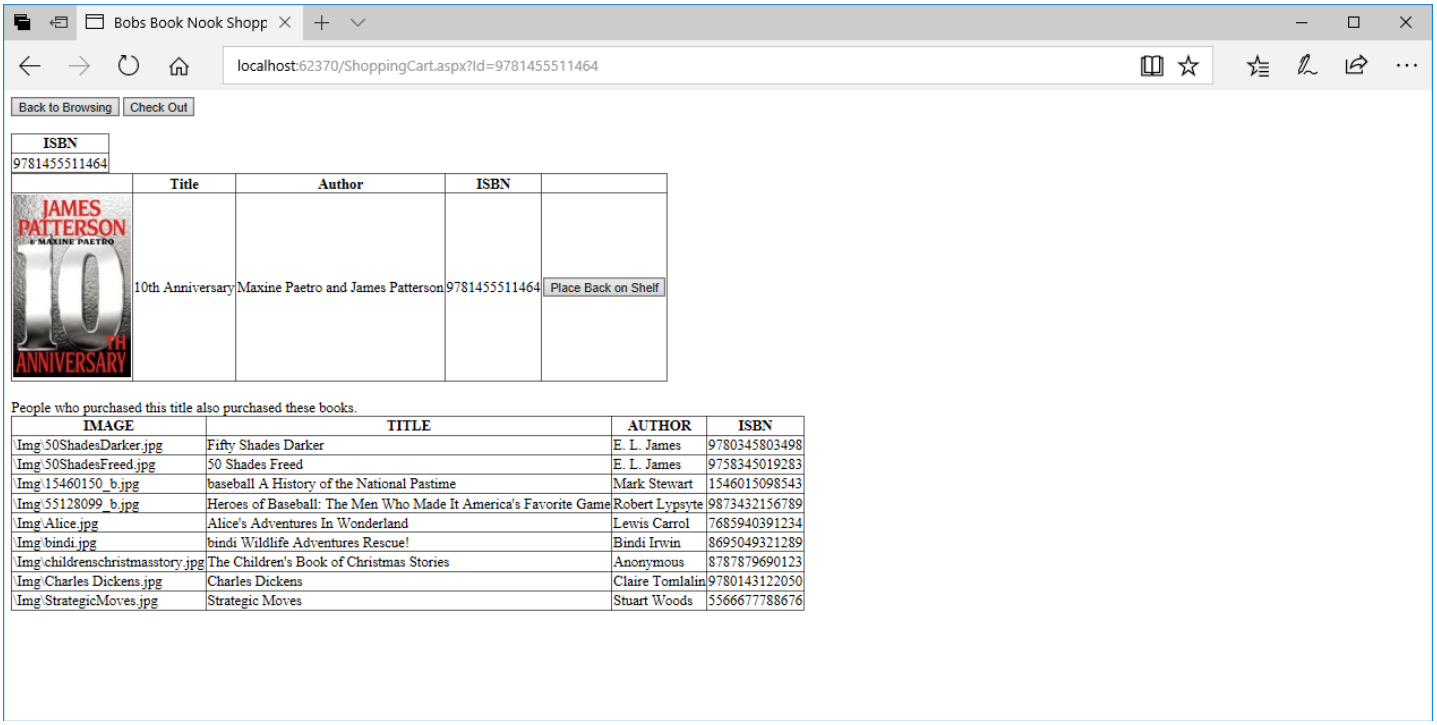



```
sqlCommand = "SELECT * FROM " + ownerID + "BOOKS WHERE " + ownerID + "BOOKS.ISBN NOT LIKE '" + ISBN +
"' AND " + ownerID + "BOOKS.ISBN IN (SELECT DISTINCT ISBN FROM " + ownerID + "HISTORY WHERE " +
ownerID + "HISTORY.EMAIL IN (SELECT DISTINCT " + ownerID + "HISTORY.EMAIL FROM " + ownerID + "HISTORY
WHERE " + ownerID + "HISTORY.ISBN = '" + ISBN + "'))";
```



And finally:

```
  sqlCommand = "SELECT IMAGE, TITLE, AUTHOR, ISBN FROM " + ownerID + "BOOKS WHERE " + ownerID +
"BOOKS.ISBN NOT LIKE '" + ISBN + "' AND " + ownerID + "BOOKS.ISBN IN (SELECT DISTINCT ISBN FROM " +
ownerID + "HISTORY WHERE " + ownerID + "HISTORY.EMAIL IN (SELECT DISTINCT " + ownerID + "HISTORY.EMAIL
FROM " + ownerID + "HISTORY WHERE " + ownerID + "HISTORY.ISBN = '" + ISBN + "'))";
```



We are now ready to include an image and then a buy me button.
First things first!

Use the gridview code for **gvShoppingCart** to modify the gridview for **gvPeopleWhoPurchasedThis**

```
<asp:GridView ID="gvShoppingCart" runat="server" AutoGenerateColumns="False" DataKeyNames="ISBN"
OnSelectedIndexChanged="gvShoppingCart_SelectedIndexChanged">
 <Columns>
  <asp:TemplateField>
   <ItemTemplate>
    <asp:Image ID="Image1" runat ="server" ImageUrl='<%# (string) FormatImageUrl( (string)
Eval("Image")) %>' />
    </ItemTemplate>
   </asp:TemplateField>
  <asp:BoundField DataField="Title" HeaderText="Title" />
  <asp:BoundField DataField="Author" HeaderText="Author" />
  <asp:BoundField DataField="ISBN" HeaderText="ISBN" />
     <asp:CommandField ButtonType="Button" InsertVisible="False" SelectText="Return to Shelf"
ShowSelectButton="True" />
 </Columns>
</asp:GridView>

  <asp:GridView ID="gvPeopleWhoPurchasedThis" runat="server">
        </asp:GridView>
```

Becomes:

```
<asp:GridView ID="gvPeopleWhoPurchasedThis" runat="server" AutoGenerateColumns="False"
DataKeyNames="ISBN">
<Columns>
  <asp:TemplateField>
   <ItemTemplate>
    <asp:Image ID="Image1" runat ="server" ImageUrl='<%# (string) FormatImageUrl( (string)
Eval("Image")) %>' />
    </ItemTemplate>
   </asp:TemplateField>
  <asp:BoundField DataField="Title" HeaderText="Title" />
  <asp:BoundField DataField="Author" HeaderText="Author" />
  <asp:BoundField DataField="ISBN" HeaderText="ISBN" />
     <asp:CommandField ButtonType="Button" InsertVisible="False" SelectText="Purchase"
ShowSelectButton="True" />
 </Columns>

</asp:GridView>
```

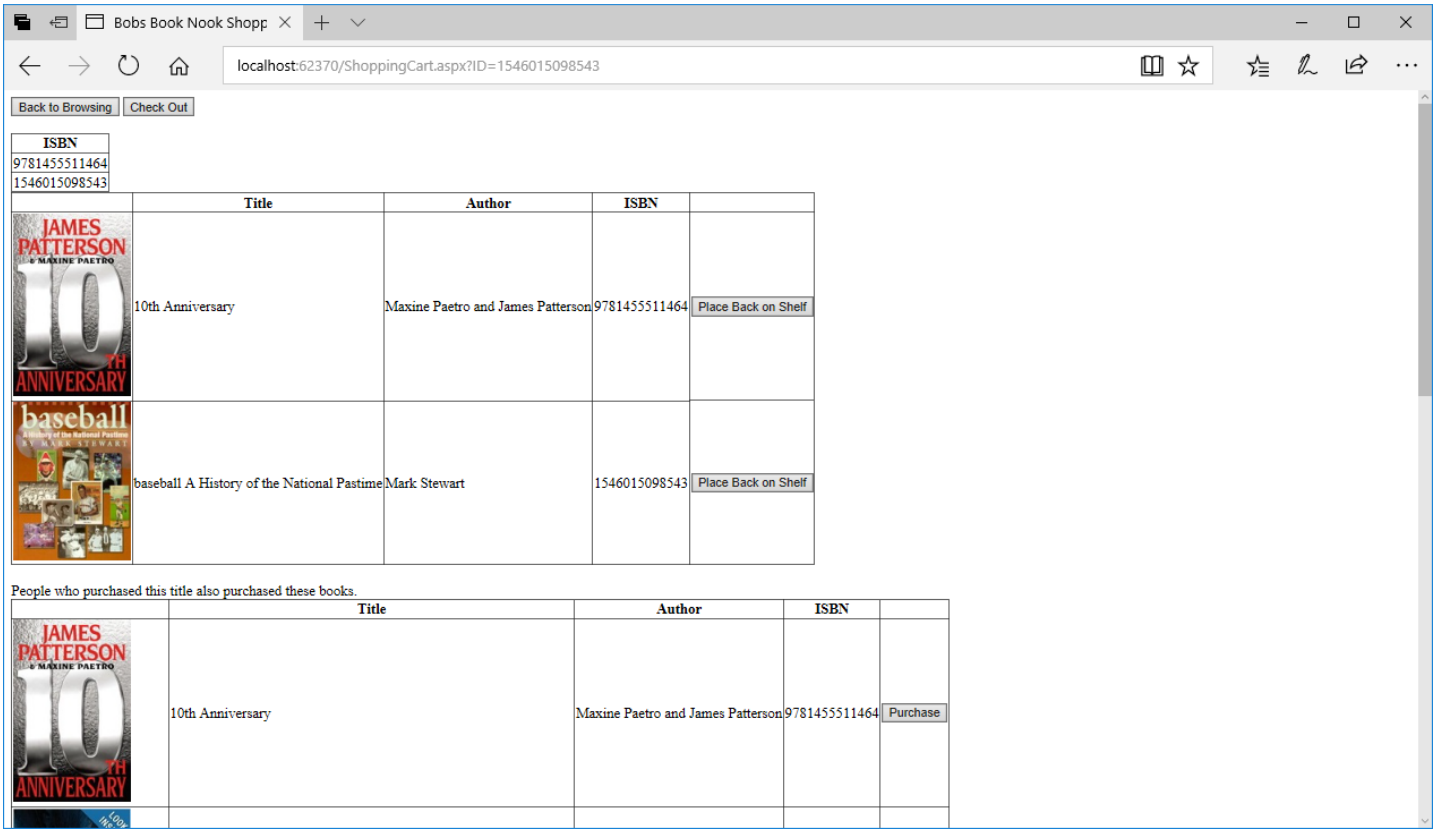
Note that we do change **="Return to Shelf"** to **="Purchase"**.

Double click on **gvPeopleWhoPurchasedThis** to create the selectedIndexChanged method.

We want to actually purchase this text, we use recursive processing to call this page, sending it the ISBN from the selected book.

```
protected void gvPeopleWhoPurchasedThis_SelectedIndexChanged(…
    {
Response.Redirect("ShoppingCart.aspx?ID=" + gvPeopleWhoPurchasedThis.SelectedValue);
    }
```

As you can see, the list will change depending on which book you select from the list.



That is it!

- Make sure you have removed any extra response.write statements used during development.
- Make sure gvHidden stays hidden. Hint: In Admin.aspx.cs, look for any myDataBaseConnection calls to gvHidden and add gvHidden.Visible = false; where appropriate.
- Make sure any other grid views that should remain hidden remain hidden.
- Make sure the program executes as expected and that you have not lost any previous additions.
- Post your URL pointing to Default.aspx.